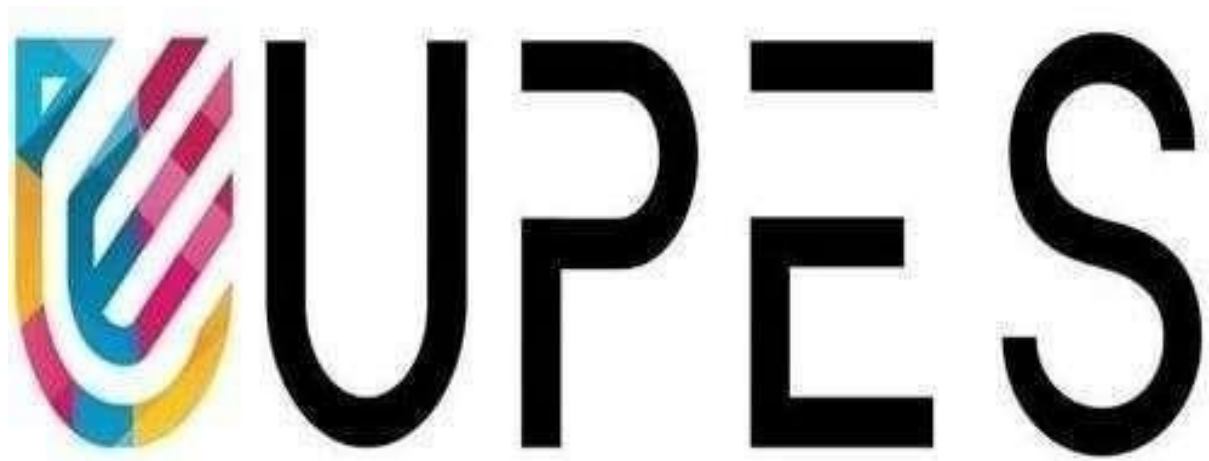




FAKE NEWS DETECTION

Submitted by –

Anshika Arjariya – 590023259

Yashika Bansal – 590024966

Batch – 48

Course – Python Programming

Course Code – CSEG1021

Supervisor – Manish Giri Sir

April 29 , 2026

Abstract

Fake news has emerged as one of the most serious threats in the digital era, influencing public opinion, elections, health decisions, and social harmony. With the exponential growth of online media platforms, manually verifying every news article has become nearly impossible.

Therefore, there is a growing demand for automated systems capable of detecting misinformation quickly and accurately.

This project presents an AI-Driven Fake News Detection System, developed as a desktop-based application using Python, Tkinter, MongoDB, and a custom implementation of the Multinomial Naive Bayes algorithm built entirely from scratch. The system classifies user-provided news content into two categories: REAL or FAKE.

The project uses CSV datasets for training, where news text is preprocessed through lowercasing, regular expression cleaning, tokenization, and stop-word removal. The classifier computes probabilistic scores using Bayes' Theorem with Laplace smoothing and logarithmic probability calculations for numerical stability.

A graphical user interface designed using Tkinter allows users to load datasets, input custom news text, obtain prediction results in real time, visualize cumulative results through graphs using Matplotlib, and store prediction history in MongoDB.

The system demonstrates how lightweight and transparent machine learning models can be effectively deployed locally without dependency on cloud services.

Contents

1. Introduction...	5
2. Problem Statement...	6
2.1 Slow Response Time	
2.2 Scalability Issues	
2.3 Existing Tool Gaps	
2.4 Need for Solution	
3. Objectives...	7
4. System Architecture...	7-9
4.1 Data Ingestion	
4.2 Preprocessing Module	
4.3 Classification Engine	
4.4 Database Layer	
4.5 User Interface	
4.6 Flowchart	
4.7 Algorithm	
5. Literature Review...	10
5.1 Machine Learning Approaches	
5.2 Deep Learning Approaches	
5.3 Importance in Digital Age	
6. Methodology...	11
6.1 Text Cleaning	
6.2 Frequency Counting	
6.3 Vocabulary Reduction	
6.4 Prediction Flow	
7. Mathematical Model...	12-13
7.1 Bayes's Theorem	
7.2 Naïve Assumption	
7.3 Laplace Smoothing	
7.4 Log Probabilities	
8. Implementation Details...	14
8.1 Technology Stack	
8.2 Major Functions in Code	
9. Results and Analysis...	15-20
9.1 User Interface Features	
9.2 Graphical Analysis	
9.3 Result Screenshot	
10. Advantages and Limitations...	21
10.1 Advantages	
10.2 Limitations	
11. Conclusions...	21

12. Future Scope.....	22
12.1 Multi-Language Support	
12.2 Web Migration	
12.3 Real-Time APIs	
12.4 Deep Learning Update	
12.5 Explainable AI	
13. Code.....	23-31
14. References... ..	32

1. Introduction

The rapid growth of the internet and social media has revolutionized communication. News can now reach millions of people instantly through platforms such as Facebook, X, YouTube, and messaging applications.

While this has democratized information sharing, it has also created an environment where misinformation spreads rapidly.

Fake news refers to deliberately misleading or false information presented as legitimate journalism. It is often created to manipulate political opinions, generate advertising revenue, spread propaganda, or create panic.

Research studies indicate that false information spreads significantly faster than truthful information online—sometimes nearly six times faster. This creates serious risks for democracy, healthcare awareness, disaster management, and public trust.

Examples include:

- False political claims during elections
- Misleading medical advice during pandemics
- Fabricated communal or social incidents
- Clickbait headlines for monetary gain

To address these challenges, intelligent automated fake news detection systems are necessary

2. Problem Statement

Traditional fact-checking depends on human experts who manually investigate sources, verify claims, and publish corrections. Although accurate, this process suffers from major limitations:

2.1 Slow Response Time

By the time fake news is manually debunked, millions of users may have already viewed or shared it.

2.2 Scalability Issues

Thousands of articles, tweets, blogs, and posts are published every minute. Human verification cannot keep pace.

2.3 Existing Tool Gaps

Many available fake news detectors are:

- Black-box systems with low transparency
- Dependent on cloud APIs
- Resource intensive
- Not suitable for offline/local execution
- Difficult for students and researchers to understand

2.4 Need for Solution

There is a need for a lightweight, transparent, low-cost, and locally hosted classifier that can quickly analyze text and predict whether a news article is real or fake.

3. Objectives

The main objectives of this project are:

1. To build a fake news detection system using machine learning.
2. To implement Naive Bayes manually without using prebuilt libraries.
3. To create an interactive GUI using Tkinter.
4. To support dataset loading from CSV files.
5. To store prediction history in MongoDB.
6. To visualize prediction statistics using graphs.
7. To provide transparent and explainable classification.

4. System Architecture

4.1 Data Ingestion

The system loads labelled datasets containing:

- text column
- label column (FAKE / REAL)

4.2 Preprocessing Module

Raw text is cleaned using regular expressions and stop-word filtering.

4.3 Classification Engine

The Naive Bayes model calculates class probabilities.

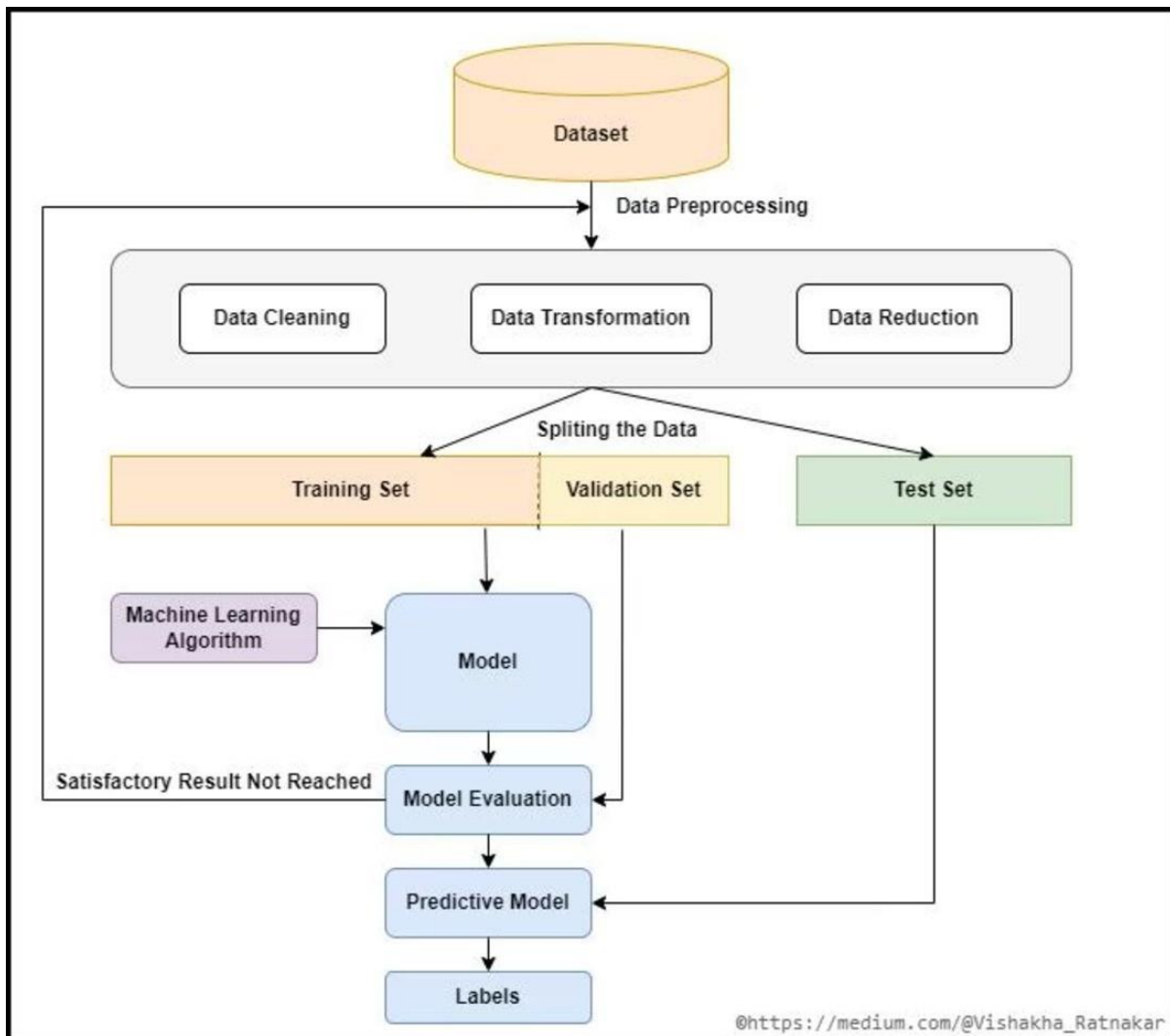
4.4 Database Layer

Predictions are stored in MongoDB.

4.5 User Interface

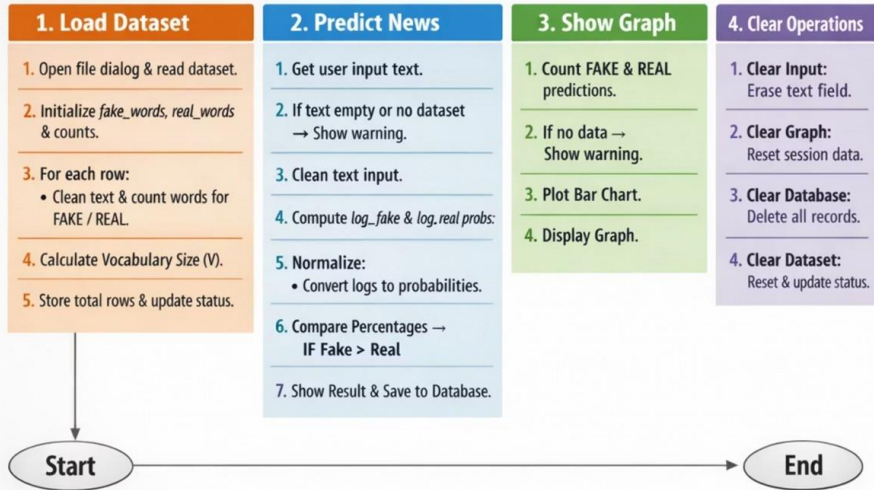
Developed using Tkinter for desktop interaction.

4.6 FLOWCHART



4.7 ALGORITHM

Algorithm: Fake News Detection System (Naive Bayes)



5. Literature Review

Automated fake news detection has become an active research area in Artificial Intelligence and Natural Language Processing.

Several techniques are commonly used:

5.1 Machine Learning Approaches

- Naive Bayes
- Logistic Regression
- Support Vector Machine
- Random Forest

These methods rely on text features and statistical patterns.

5.2 Deep Learning Approaches

- Recurrent Neural Networks
- LSTM Networks
- Transformer Models
- BERT

These models offer higher accuracy but require powerful hardware and large datasets.

5.3 Importance in Digital Age

Automated detection systems are important because they:

- Protect information integrity
- Reduce public manipulation
- Improve trust in journalism
- Assist moderation systems
- Enable early warning systems

For this project, “Naive Bayes” was selected because it is simple, efficient, interpretable, and suitable for academic demonstration.

6. Methodology

6.1 Text Cleaning

The function performs:

- Convert to lowercase
- Remove special symbols
- Remove numbers
- Tokenization
- Stop-word removal

6.2 Frequency Counting

The system stores word frequencies separately for:

- Fake news words
- Real news words

6.3 Vocabulary Reduction

Top 1000 most frequent words from each class are retained for efficient memory usage.

6.4 Prediction Flow

When user enters text:

1. Clean text
2. Tokenize words
3. Calculate log probability for Fake
4. Calculate log probability for Real
5. Compare results
6. Display confidence percentage

7. Mathematical Model

7.1 Bayes' Theorem

The fundamental formula used is:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Posterior = useful evidence / total evidence Where:

- $P(C|X)$ = Posterior Probability (Probability of class C given text X)
- $P(X|C)$ = Likelihood (Probability of text X appearing in class C)
- $P(C)$ = Prior Probability of class C
- $P(X)$ = Probability of observing text X
- C = Class label (FAKE or REAL)
- X = Input news text

The system compares probabilities for both classes:

- $P(\text{FAKE} | X)$
- $P(\text{REAL} | X)$

Whichever is higher becomes the predicted result.

7.2 Naive Assumption

Naive Bayes assumes that each word in the sentence is independent of the others.

For example:

"breaking news today" The

probability becomes:

$$P(X|C) = P(w_1|C) \cdot P(w_2|C) \cdot P(w_3|C)$$

Where:

- $w_1 = \text{breaking}$; $w_2 = \text{news}$; $w_3 = \text{today}$

This assumption simplifies calculations and makes the algorithm fast.

7.3 Laplace Smoothing

To handle unseen words:

$$P(w|C) = \frac{\text{count}(w,C)+1}{\text{Total}(C)+V}$$

Where:

- $V = \text{Vocabulary size}$

7.4 Log Probabilities

Instead of multiplying many small probabilities:

$$\log(ab) = \log a + \log b$$

This prevents arithmetic underflow.

8. Implementation Details

8.1 *Technology Stack*

Component	-	Technology
Programming Language	-	Python 3
GUI	-	Tkinter
Database	-	MongoDB
Visualization	-	Matplotlib
Data Handling	-	Pandas

8.2 *Major Functions in Code*

`load_dataset()` -

- Opens file chooser
- Loads CSV/Excel
- Trains word frequency dictionaries

`clean_text()` -

- Cleans and tokenizes text

`predict_news()` -

- Accepts user text
- Computes class probabilities

- Displays REAL or FAKE
- show_graph() -
- Generates prediction distribution chart
- show_data() -
- Displays stored MongoDB records

9. Results and Analysis

9.1 User Interface Features

The developed GUI includes:

- Dataset loading button
- Text area for entering news
- Analyse News button
- Result label with confidence
- Graph button
- Data history button
- Clear utilities

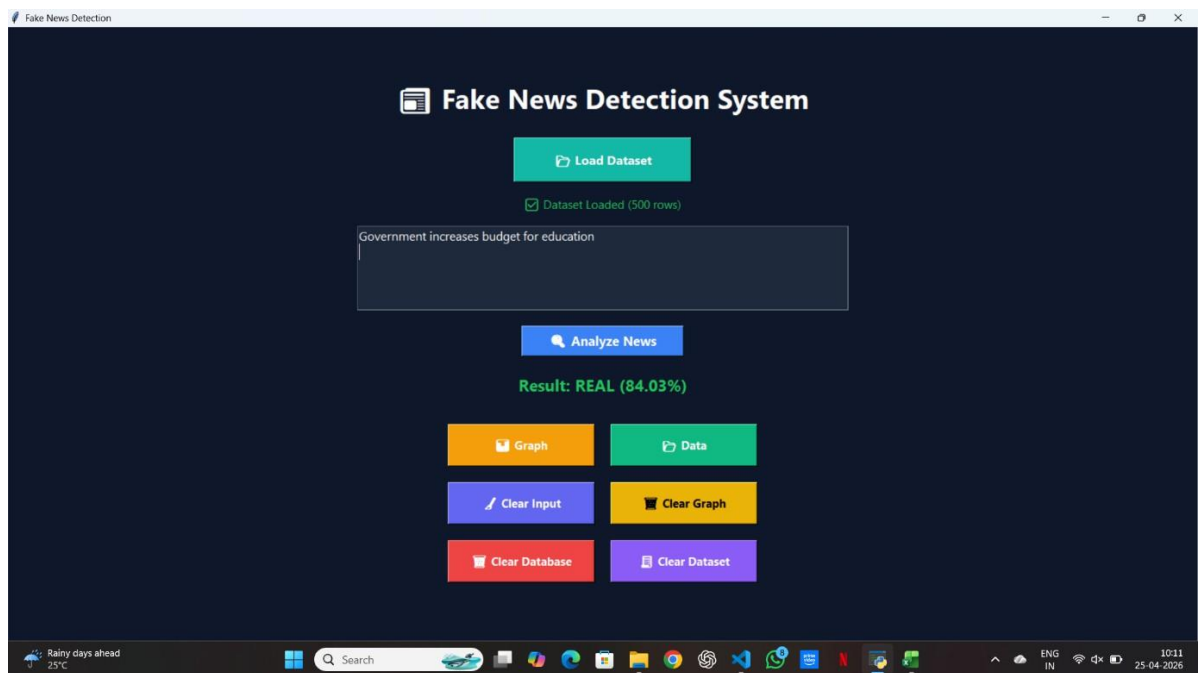
9.2 Graphical Analysis

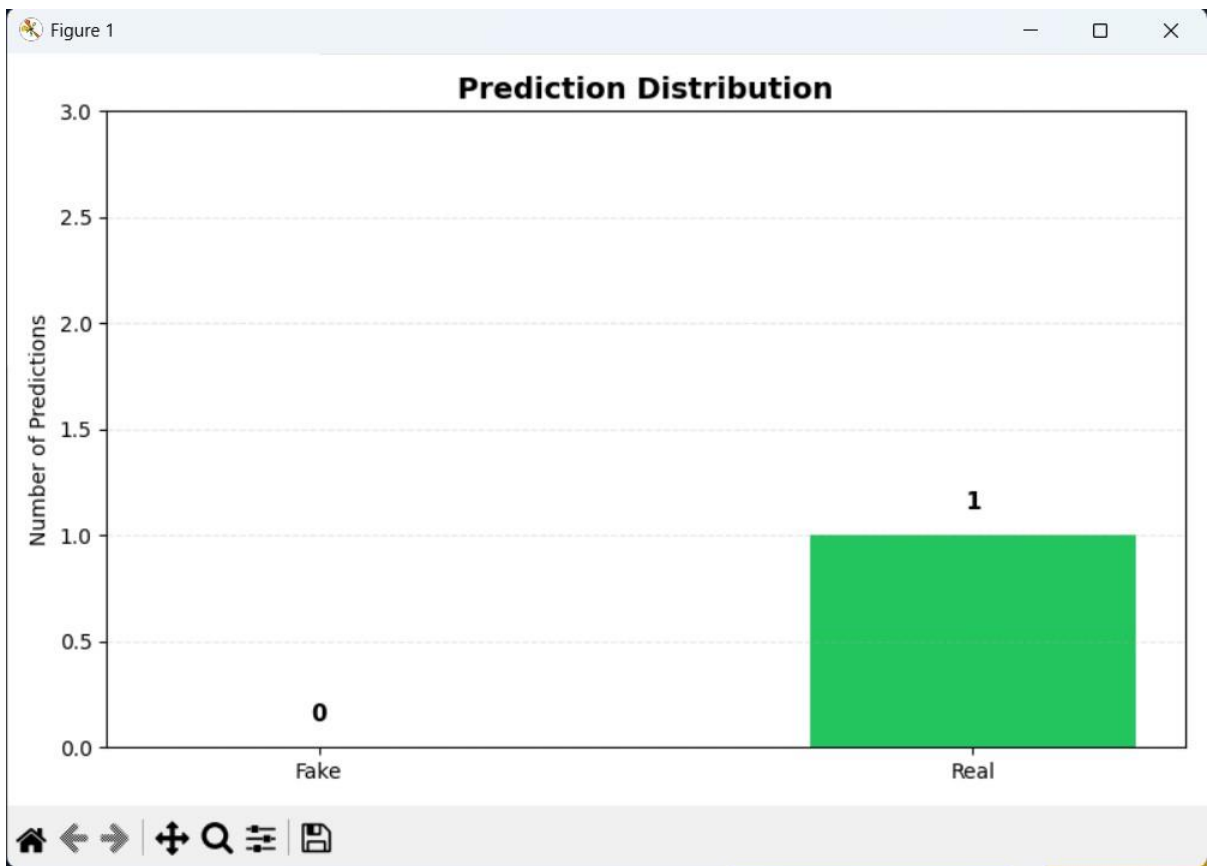
Using Matplotlib, bar charts show:

- Total Fake Predictions
- Total Real Predictions

This helps understand usage trends.

9.3 Result Screenshots





Stored Data

Government increases budget for education → REAL

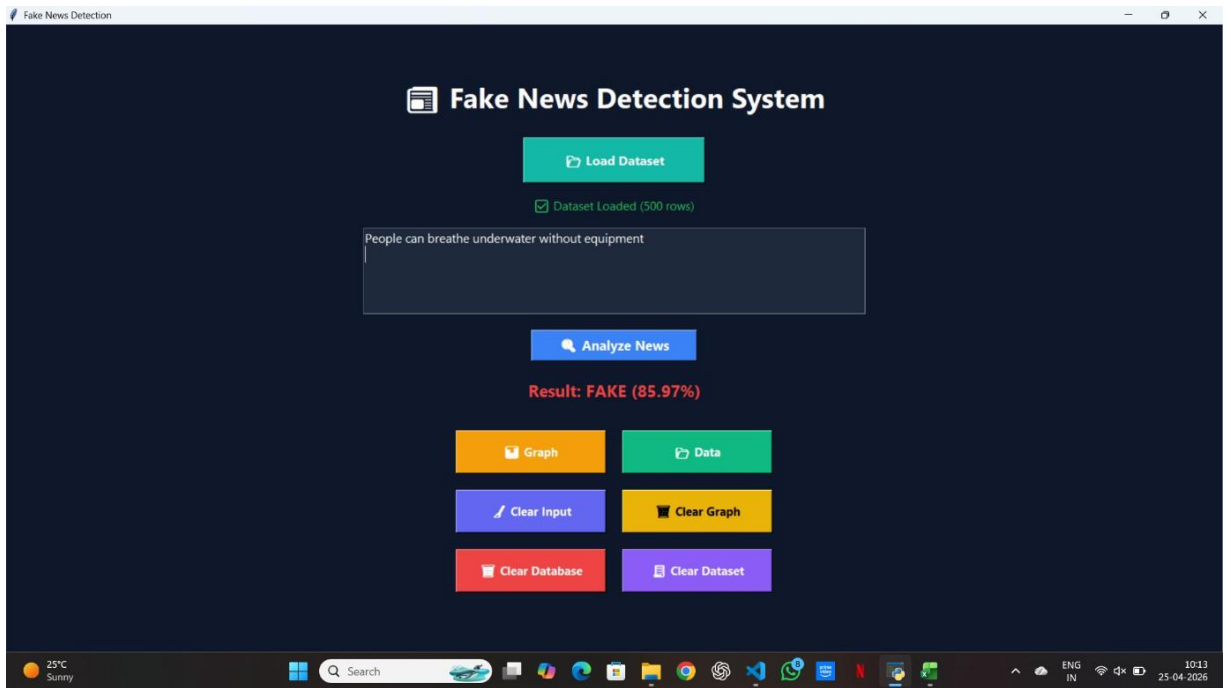
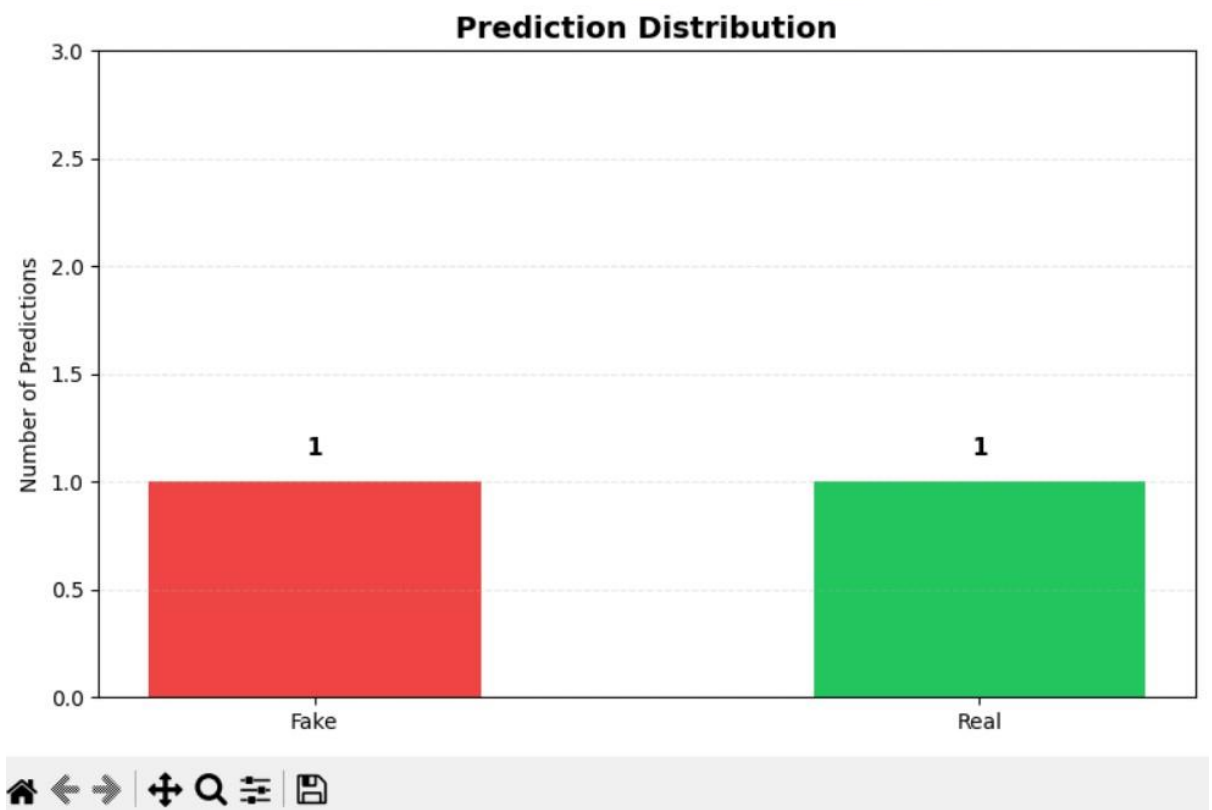
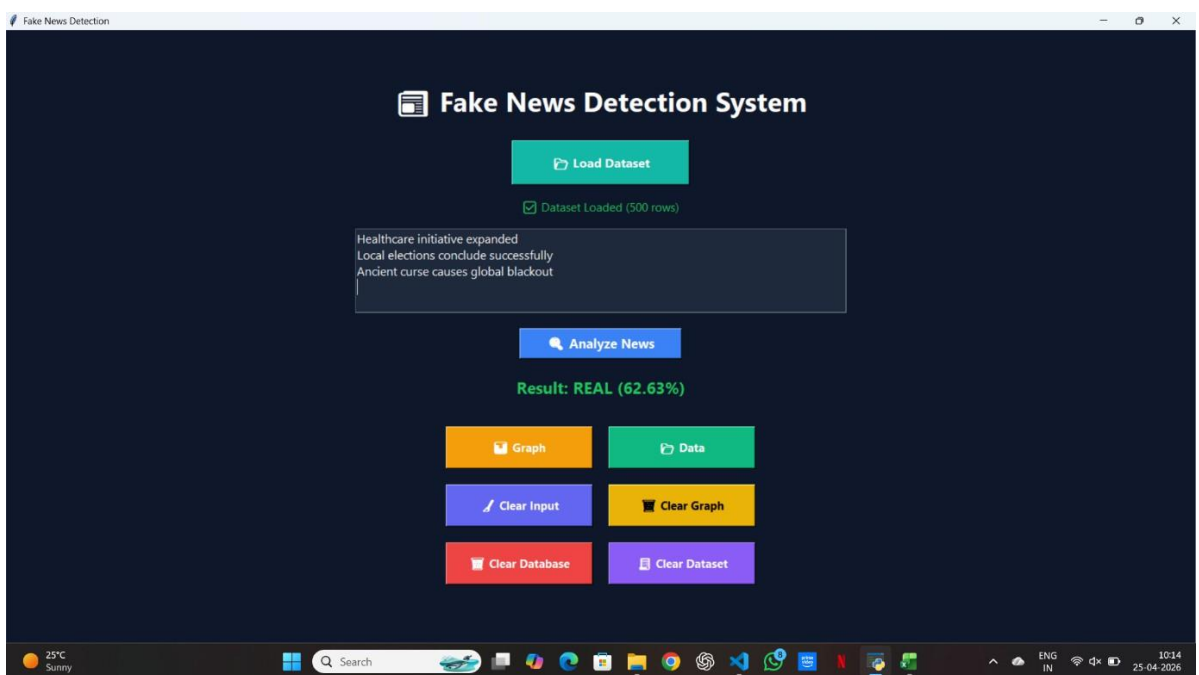
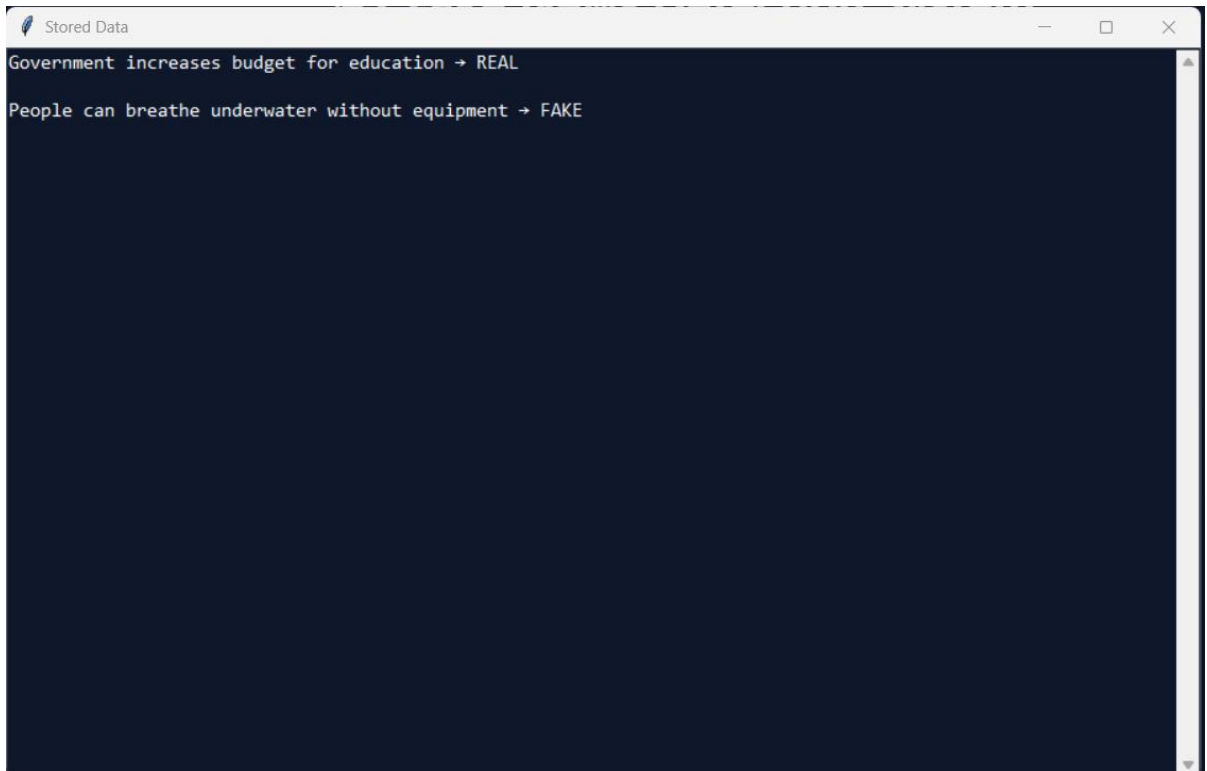
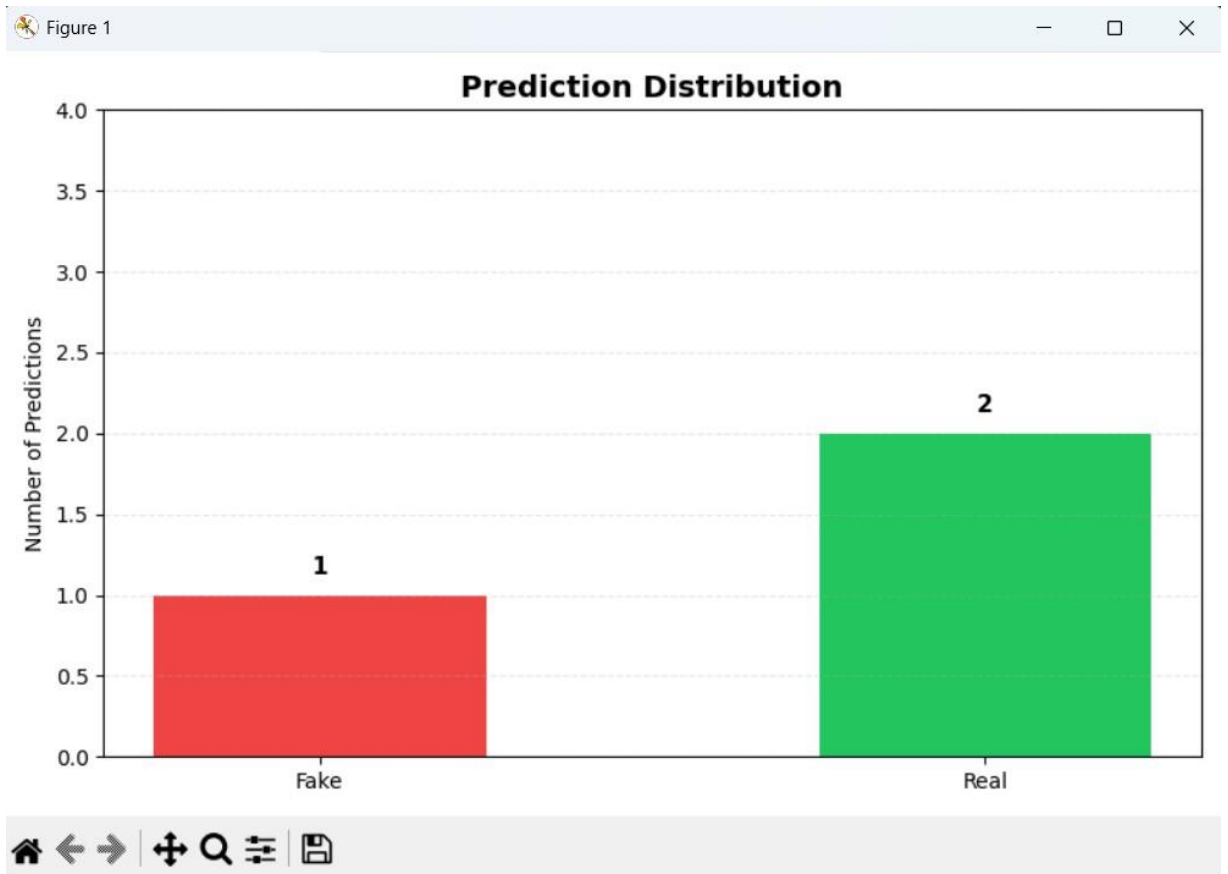


Figure 1







Stored Data

```
Government increases budget for education → REAL
People can breathe underwater without equipment → FAKE
Healthcare initiative expanded
Local elections conclude successfully
Ancient curse causes global blackout → REAL
```

10. Advantages and Limitations

a. Advantages

- i. Fast predictions
- ii. Lightweight system
- iii. Transparent algorithm
- iv. Offline/local execution
- v. Easy GUI usage
- vi. Database integration

b. Limitations

- i. Depends on dataset quality
- ii. Cannot understand sarcasm/context deeply
- iii. Lower accuracy than transformer models
- iv. English-only preprocessing currently

11. Conclusion

This project successfully developed a fully functional AI-Driven Fake News Detection System using machine learning concepts and practical software tools.

A custom implementation of Multinomial Naive Bayes was created from scratch, proving that effective text classification can be achieved without relying entirely on external ML libraries. Integration with Tkinter and MongoDB transformed the classifier into a usable desktop application.

The project demonstrates an important milestone in combining:

- Artificial Intelligence
- Natural Language Processing
- GUI Development
- Database Systems
- Data Visualization

12. Future Scope

Future enhancements may include:

a. Multi-language Support

Support for:

- i. Hindi
- ii. Urdu
- iii. Punjabi
- iv. Regional Indian scripts

b. Web Migration

Deploy system using:

- i. Flask
- ii. Streamlit

c. *Real-Time APIs*

Integrate live feeds from:

- i. News websites
- ii. Social media streams
- iii. RSS feeds

d. *Deep Learning Upgrade*

Move toward advanced NLP models:

- i. BERT
- ii. RoBERTa
- iii. DistilBERT

e. *Explainable AI*

Show why prediction was FAKE/REAL using top keywords.

13. Code

```
fake_news_detection.py > ...
1  import tkinter as tk
2  from tkinter import filedialog, messagebox
3  import pandas as pd
4  import re
5  import math
6  from collections import defaultdict, Counter
7  import matplotlib.pyplot as plt
8  from pymongo import MongoClient
9
10 # ----- DATABASE -----
11 client = MongoClient("mongodb://localhost:27017/")
12 db = client["fake_news_db"]
13 collection = db["predictions"]
14
15 # ----- GLOBALS -----
16 fake_words = defaultdict(int)
17 real_words = defaultdict(int)
18 fake_total = 0
19 real_total = 0
20 v = 0
21 df = None
22 row_count = 0
23 session_data = []
24
25 # ----- STOP WORDS -----
26 stop_words = {
27     "the", "is", "in", "at", "a", "an", "and", "to", "of", "for", "on", "with",
28     "that", "this", "it", "as", "by", "from", "was", "are", "be"
29 }
30
31 # ----- CLEAN TEXT -----
32 def clean_text(text):
33     text = text.lower()
34     text = re.sub(r'^a-z\s', '', text)
35     return [w for w in text.split() if w not in stop_words]
36
```



```

37 # ----- LOAD DATASET -----
38 def load_dataset():
39     global df, fake_words, real_words, fake_total, real_total, v, row_count
40
41     file_path = filedialog.askopenfilename(
42         |     filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx")]
43     )
44
45     if not file_path:
46         |     return
47
48     try:
49         |     if file_path.endswith(".csv"):
50             |         df = pd.read_csv(file_path)
51         |     else:
52             |         df = pd.read_excel(file_path)
53
54         |     fake_words.clear()
55         |     real_words.clear()
56         |     fake_total = 0
57         |     real_total = 0
58
59         |     for _, row in df.iterrows():
60             |         words = clean_text(str(row['text']))
61
62             |         if row['label'] == "FAKE":
63                 |             for w in words:
64                     |                 fake_words[w] += 1
65                     |                 fake_total += 1
66             |         else:
67                 |             for w in words:
68                     |                 real_words[w] += 1
69                     |                 real_total += 1
70

```

```

70
71     # ✅ VOCABULARY LIMIT (Top 1000 words)
72     top_fake = dict(Counter(fake_words).most_common(1000))
73     top_real = dict(Counter(real_words).most_common(1000))
74
75     fake_words.clear()
76     real_words.clear()
77
78     fake_words.update(top_fake)
79     real_words.update(top_real)
80
81     v = len(set(list(fake_words.keys()) + list(real_words.keys())))
82     row_count = len(df)
83
84     status_label.config(
85         |     text=f"✅ Dataset Loaded ({row_count} rows)",
86         |     fg="✅"#22c55e"
87     )
88
89     except Exception as e:
90         |     messagebox.showerror("Error", str(e))
91
92     # ----- PREDICT (IMPROVED) -----
93     def predict_news():
94         |     global session_data
95
96         |     text = entry.get("1.0", tk.END).strip()
97
98         |     if text == "":
99             |         messagebox.showwarning("Warning", "Enter news text")
100            |         return
101
102         |     if fake_total == 0 or real_total == 0:
103             |         messagebox.showwarning("Warning", "Load dataset first!")
104            |         return
105

```

```

106 words = clean_text(text)
107
108 # ✅ Better priors (Laplace smoothing)
109 log_fake = math.log((fake_total + 1) / (fake_total + real_total + 2))
110 log_real = math.log((real_total + 1) / (fake_total + real_total + 2))
111
112 for w in words:
113     log_fake += math.log((fake_words[w] + 1) / (fake_total + V))
114     log_real += math.log((real_words[w] + 1) / (real_total + V))
115
116 # ✅ Length normalization
117 if len(words) > 0:
118     log_fake /= len(words)
119     log_real /= len(words)
120
121 max_log = max(log_fake, log_real)
122
123 # ✅ Temperature smoothing
124 temperature = 2.0
125
126 fake_prob = math.exp((log_fake - max_log) / temperature)
127 real_prob = math.exp((log_real - max_log) / temperature)
128
129 total = fake_prob + real_prob
130
131 fake_percent = (fake_prob / total) * 100
132 real_percent = (real_prob / total) * 100
133
134 if fake_percent > real_percent:
135     result = "FAKE"
136     confidence = fake_percent
137     color = "🔴 #ef4444"
138 else:
139     result = "REAL"
140     confidence = real_percent
141     color = "🟢 #22c55e"

```

```

142
143     session_data.append(result)
144
145     result_label.config(
146         text=f"Result: {result} ({confidence:.2f}%)",
147         fg=color
148     )
149
150     collection.insert_one({"text": text, "result": result})
151
152 # ----- GRAPH -----
153 def show_graph():
154     fake_count = session_data.count("FAKE")
155     real_count = session_data.count("REAL")
156
157     total = fake_count + real_count
158     if total == 0:
159         messagebox.showwarning("No Data", "No predictions yet!")
160         return
161
162     plt.figure(figsize=(8,5))
163
164     bars = plt.bar(
165         ["Fake", "Real"],
166         [fake_count, real_count],
167         color=[ "#ef4444", "#22c55e"],
168         width=0.5
169     )
170
171     plt.title("Prediction Distribution", fontsize=14, fontweight="bold")
172     plt.ylabel("Number of Predictions")
173
174     for bar in bars:
175         height = bar.get_height()
176         plt.text(
177             bar.get_x() + bar.get_width()/2,

```

```

153 def show_graph():
178     height + 0.1,
179     f"{int(height)}",
180     ha='center',
181     va='bottom',
182     fontsize=11,
183     fontweight='bold'
184 )
185
186 plt.ylim(0, max(fake_count, real_count) + 2)
187 plt.grid(axis="y", linestyle="--", alpha=0.3)
188
189 plt.tight_layout()
190 plt.show()
191
192 # ----- SHOW DATA -----
193 def show_data():
194     win = tk.Toplevel(root)
195     win.title("Stored Data")
196     win.geometry("900x550+250+120")
197     win.configure(bg="#1e293b")
198
199     text_area = tk.Text(win, bg="#0f172a", fg="white", font=("Consolas", 11))
200     text_area.pack(fill="both", expand=True)
201
202     scrollbar = tk.Scrollbar(text_area)
203     scrollbar.pack(side="right", fill="y")
204
205     text_area.config(yscrollcommand=scrollbar.set)
206     scrollbar.config(command=text_area.yview)
207
208     for d in collection.find():
209         text_area.insert(tk.END, f"{d['text']} → {d['result']}\n\n")
210

```



```

211 # ----- CLEAR FUNCTIONS -----
212 def clear_input():
213     entry.delete("1.0", tk.END)
214     result_label.config(text="")
215
216 def clear_graph():
217     global session_data
218     session_data = []
219     messagebox.showinfo("Cleared", "Graph data cleared")
220
221 def clear_database():
222     collection.delete_many({})
223     messagebox.showinfo("Cleared", "Database cleared")
224
225 def clear_dataset():
226     global df, fake_words, real_words, fake_total, real_total, v, row_count
227
228     df = None
229     fake_words.clear()
230     real_words.clear()
231     fake_total = 0
232     real_total = 0
233     v = 0
234     row_count = 0
235
236     status_label.config(
237         text="⚠ Dataset cleared",
238         fg="■ #f59e0b"
239     )
240

```

```

241 # ----- UI -----
242 root = tk.Tk()
243 root.title("Fake News Detection")
244 root.state("zoomed")
245 root.configure(bg="#0f172a")
246
247 main_frame = tk.Frame(root, bg="#0f172a")
248 main_frame.pack(expand=True)
249
250 title = tk.Label(main_frame, text="📰 Fake News Detection System",
251                 font=("Segoe UI", 26, "bold"),
252                 fg="white", bg="#0f172a")
253 title.pack(pady=15)
254
255 tk.Button(main_frame, text="📁 Load Dataset",
256          command=load_dataset,
257          bg="#14b8a6", fg="white",
258          font=("Segoe UI", 12, "bold"),
259          width=22, height=2).pack(pady=10)
260
261 status_label = tk.Label(main_frame, text="No dataset loaded",
262                        fg="#ef4444", bg="#0f172a",
263                        font=("Segoe UI", 11))
264 status_label.pack(pady=5)
265
266 entry = tk.Text(main_frame, height=5, width=70,
267                font=("Segoe UI", 12),
268                bg="#1e293b", fg="white",
269                insertbackground="white")
270 entry.pack(pady=10)
271
272 tk.Button(main_frame, text="🔍 Analyze News",
273          command=predict_news,
274          bg="#3b82f6", fg="white",
275          font=("Segoe UI", 13, "bold"),
276          width=20, height=1).pack(pady=10)
277

```

```

278 result_label = tk.Label(main_frame, text="",
279 | | | | | font=("Segoe UI", 16, "bold"),
280 | | | | | bg=□ "#0f172a")
281 result_label.pack(pady=8)
282
283 btn_frame = tk.Frame(main_frame, bg=□ "#0f172a")
284 btn_frame.pack(pady=15)
285
286 btn_style = {"width":20, "height":2, "font":("Segoe UI", 11, "bold")}
287
288 tk.Button(btn_frame, text="📊 Graph", bg=■ "#f59e0b", fg="white",
289 | | | | | command=show_graph, **btn_style).grid(row=0, column=0, padx=10, pady=10)
290
291 tk.Button(btn_frame, text="📁 Data", bg=■ "#10b981", fg="white",
292 | | | | | command=show_data, **btn_style).grid(row=0, column=1, padx=10, pady=10)
293
294 tk.Button(btn_frame, text="✂ Clear Input", bg=■ "#6366f1", fg="white",
295 | | | | | command=clear_input, **btn_style).grid(row=1, column=0, padx=10, pady=10)
296
297 tk.Button(btn_frame, text="🗑 Clear Graph", bg=■ "#eab308", fg="black",
298 | | | | | command=clear_graph, **btn_style).grid(row=1, column=1, padx=10, pady=10)
299
300 tk.Button(btn_frame, text="🗑 Clear Database", bg=■ "#ef4444", fg="white",
301 | | | | | command=clear_database, **btn_style).grid(row=2, column=0, padx=10, pady=10)
302
303 tk.Button(btn_frame, text="🗑 Clear Dataset", bg=■ "#8b5cf6", fg="white",
304 | | | | | command=clear_dataset, **btn_style).grid(row=2, column=1, padx=10, pady=10)
305
306 root.mainloop()

```


14. References

- Fake News Dataset Collections
- GitHub Open Source Fake News Projects
- Christopher D. Manning – Introduction to Information Retrieval
- Tom Mitchell – Machine Learning
- Research papers on misinformation detection and NLP classification methods.